Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin

GOOGLE BRAIN, GOOGLE RESEARCH, UNIVERSITY OF TORONTO

The Challenge of Sequential Models

Dominant models like RNNs and LSTMs are inherently sequential, processing data token-by-token.

This sequential nature (h_t depends on h_{t-1}) prevents parallelization within examples.

Learning long-range dependencies is a key challenge due to long signal paths and vanishing gradients.

Memory constraints limit batching across examples for long sequences.

A New Architecture: The Transformer

We propose a network architecture based solely on attention mechanisms.

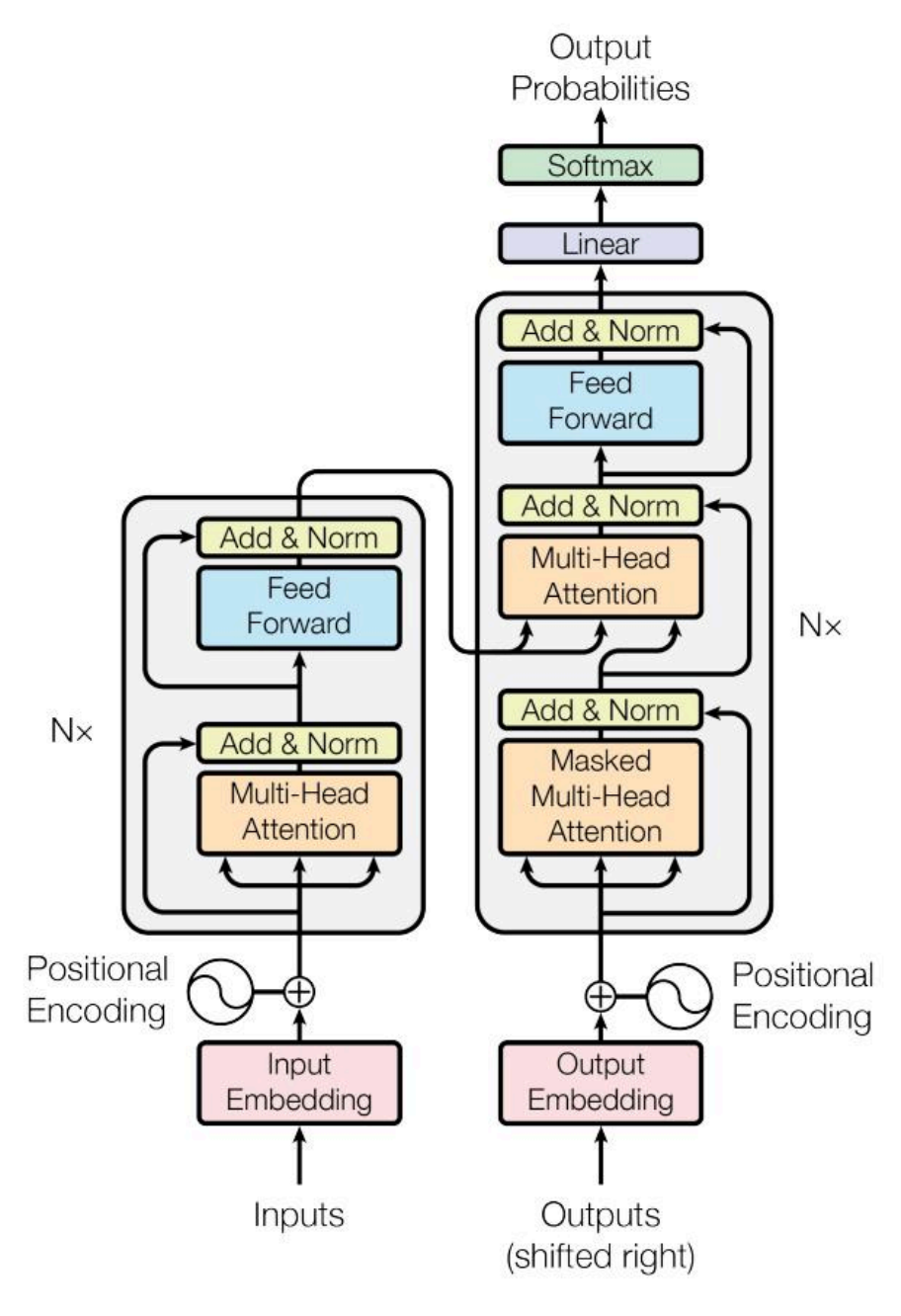
It dispenses with recurrence and convolutions entirely.

This design allows for significantly more parallelization, reducing training time.

Achieves state-of-the-art translation quality while requiring less time to train.

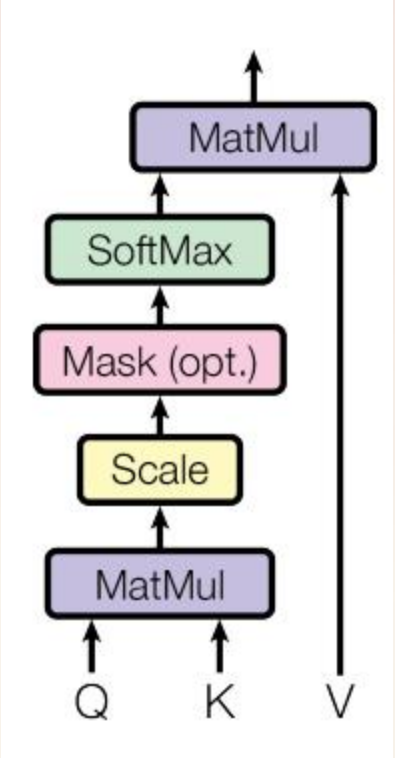
High-Level Architecture

- The Transformer follows a standard encoder-decoder structure.
- **Encoder:** Maps an input sequence of symbols to a sequence of continuous representations.
- **Decoder:** Generates an output sequence one symbol at a time, consuming previously generated symbols.
- Both components use stacked self-attention and point-wise, fully connected layers.



The Core Mechanism: Attention

- An attention function maps a query and a set of key-value pairs to an output.
- The output is a weighted sum of the values, where weights are computed based on query-key compatibility.
- We use a specific implementation called Scaled Dot-Product Attention.



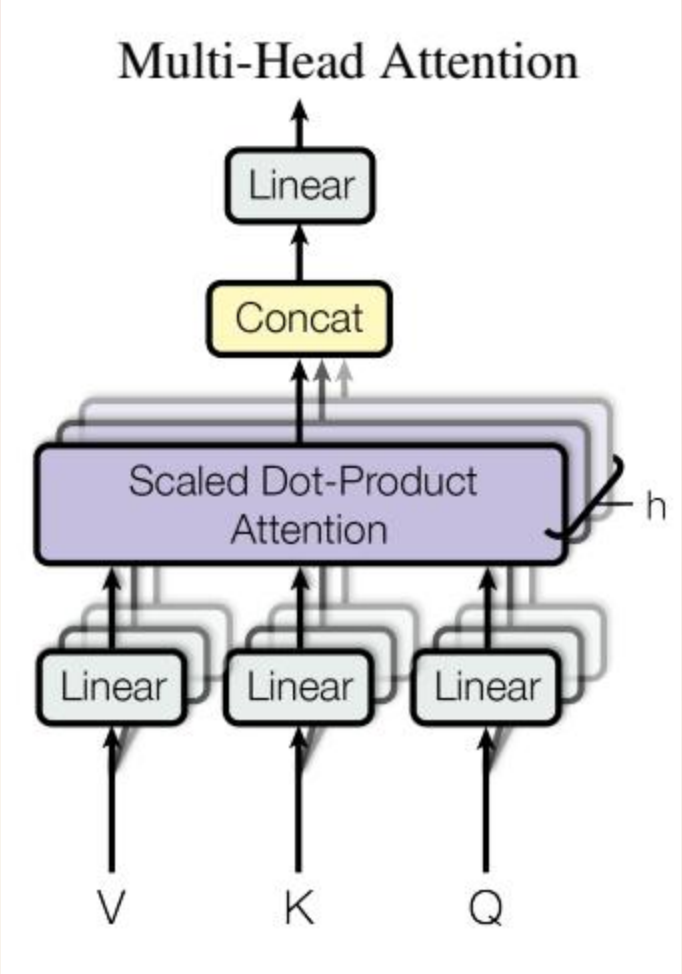
Scaled Dot-Product Attention

- Input consists of queries (Q), keys (K) of dimension d_k, and values (V) of dimension d_v.
- Dot products of the query with all keys are computed, then scaled by $\frac{1}{\sqrt{d_k}}$.
- Scaling prevents the softmax function from entering regions with small gradients, stabilizing training.
- A softmax is applied to obtain weights on the values.

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

Multi-Head Attention

- Instead of a single attention function, we use multiple attention 'heads' in parallel.
- Queries, keys, and values are linearly projected h times with different learned projections.
- This allows the model to jointly attend to information from different representation subspaces at different positions.
- Outputs from each head are concatenated and projected to form the final result.



Multi-Head Attention in Detail

- We use $h = 8$ parallel attention heads.
- For each head, we use reduced key, value, and query dimensions: $d_k = d_v = d_{model}/h = 64$.
- The total computational cost is similar to single-head attention with full dimensionality.

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \\ \text{where head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned}$$

Layer Structure

Encoder Layer

- Composed of two sub-layers:

  i. Multi-Head Self-Attention
  ii. Position-wise Feed-Forward Network

- Residual connections and Layer Normalization are applied around each sub-layer.

Decoder Layer

- Composed of three sub-layers:

  i. Masked Multi-Head Self-Attention
  ii. Encoder-Decoder Attention
  iii. Position-wise Feed-Forward Network

- Masking in the self-attention layer preserves the auto-regressive property.

Positional Encoding

- Since the model contains no recurrence or convolution, we must inject information about token position.
- We add 'positional encodings' to the input embeddings at the bottom of the encoder and decoder stacks.
- We use sine and cosine functions of different frequencies.

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

Why Self-Attention?

- Self-attention layers offer advantages in computational complexity, parallelization, and learning long-range dependencies.
- The path length between any two positions is constant ($O(1)$), unlike in recurrent ($O(n)$) or convolutional ($O(\log_k(n))$) layers.

Layer Type	*Complexity per Layer*	*Sequential Operations*	*Maximum Path Length*
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

State-of-the-Art Machine Translation

- The Transformer outperforms the best previously reported models on WMT 2014 translation tasks.
- The big model establishes a new SOTA BLEU score of 28.4 on English-to-German.
- This performance is achieved at a fraction of the training cost of competing models.

Model	*BLEU (EN-DE)*	*BLEU (EN-FR)*	*Training Cost (FLOPs)*
GNMT + RL [38]	24.6	39.92	2.3×10^{19}
ConvS2S [9]	25.16	40.46	9.6×10^{18}
GNMT + RL Ensemble [38]	26.30	41.16	1.8×10^{20}
Transformer (base)	**27.3**	**38.1**	$\mathbf{3.3 \times 10^{18}}$
Transformer (big)	**28.4**	**41.8**	$\mathbf{2.3 \times 10^{19}}$

Ablation Studies

- We varied components to measure their importance on English-to-German translation.
- (A) 8 attention heads were found to be optimal.
- (D) Dropout is very helpful in avoiding over-fitting.
- (E) Sinusoidal positional encodings perform nearly identically to learned embeddings.

Variation	*PPL (dev)*	*BLEU (dev)*
base (h=8)	**4.92**	**25.8**
(A) h=1	5.29	24.9
(A) h=16	4.91	25.8
(D) P_{drop}=0.0	5.77	24.6
(E) Learned Positional Embedding	4.92	25.7
big	**4.33**	**26.4**

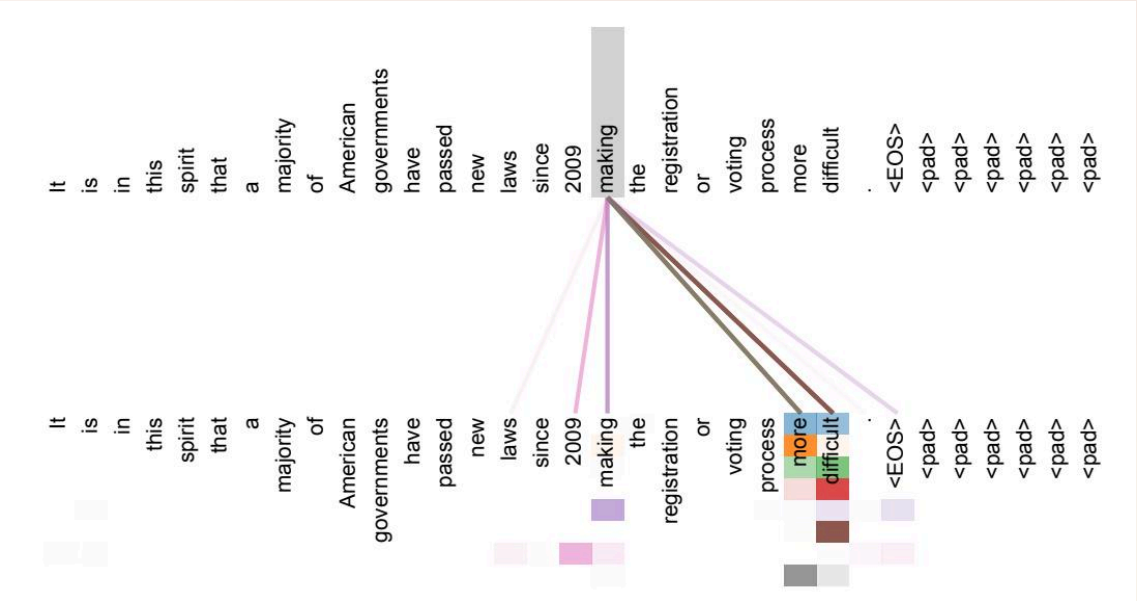
Generalizing to Other Tasks: Parsing

- To test generalization, we applied the Transformer to English constituency parsing.
- The model performs surprisingly well with minimal task-specific tuning.
- It outperforms the BerkeleyParser even when trained only on the small 40K sentence WSJ dataset.

Parser	*Training Setting*	*WSJ 23 F1 Score*
Recurrent Neural Network Grammar	generative	93.3
Vinyals & Kaiser et al. (2015)	semi-supervised	92.1
Transformer (4 layers)	**semi-supervised**	**92.7**
Dyer et al. (2016)	WSJ only	91.7
Transformer (4 layers)	**WSJ only**	**91.3**

Visualizing Attention

- Self-attention mechanisms can lead to more interpretable models.
- We can inspect attention distributions to see how the model represents syntactic and semantic structures.
- Different attention heads learn to perform different tasks, such as tracking long-distance dependencies or resolving anaphora.



Conclusion

- We presented the Transformer, the first sequence transduction model based entirely on attention.
- It trains significantly faster and achieves a new state of the art on machine translation tasks.
- We are excited about the future of attention-based models and plan to apply them to other tasks and modalities.